COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Desarrollo de Interfaces/Programación de Servicios			CURSO:	2º
	PROTOCOLO .	Apuntes clases	AVAL:		DATA:	
	Autor	Francisco M. Bellas Aláez (aka Curro)				

Elementos básicos de C#

Namespaces (Espacios de nombres):

Es una forma de agrupar los diferentes elementos (clases, interfaces, módulos, estructuras…) de forma lógica y jerarquizada.

Ensamblados

Son bloques ejecutables. Es decir son los clásicos .exe y .dll (en Windows) que componen una aplicación.

Hola Mundo en C#

```
// Importación de librerías
using System;

// Espacio de nombres → Contenedor de elementos (clases, interfaces…) similar a package Java
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Comentarios

Línea: `// Comentario`

Bloque: `/* Comentario */`

Documentación: `/// Documentación en XML (estilo javadoc)`

Tipos de datos

Tipo CTS	Descripción	Bits	Rango de valores	Alias
SByte	*Bytes* con signo	8	[-128, 127]	**sbyte**
Byte	*Bytes* sin signo	8	[0, 255]	**byte**
Int16	Enteros cortos con signo	16	[-32.768, 32.767]	**short**
UInt16	Enteros cortos sin signo	16	[0, 65.535]	**ushort**
Int32	Enteros normales	32	[-2.147.483.648, 2.147.483.647]	**int**
UInt32	Enteros normales sin signo	32	[0, 4.294.967.295]	**uint**
Int64	Enteros largos	64	[-9.223.372.036.854.775.808, 9.223.372.036.854.775.807]	**long**
UInt64	Enteros largos sin signo	64	[0-18.446.744.073.709.551.615]	**ulong**
Single	Reales con 7 dígitos de precisión	32	$[1,5\times10^{-45}$ - $3,4\times10^{38}]$	**float**
Double	Reales de 15-16 dígitos de precisión	64	$[5,0\times10^{-324}$ - $1,7\times10^{308}]$	**double**
Decimal	Reales de 28-29 dígitos de precisión	128	$[1,0\times10^{-28}$ - $7,9\times10^{28}]$	**decimal**
Boolean	Valores lógicos	32	**true**, **false**	**bool**
Char	Caracteres Unicode	16	[‘\u0000’, ‘\uFFFF’]	**char**
String	Cadenas de caracteres	Variable	El permitido por la memoria	**string**
Object	Cualquier objeto	Variable	Cualquier objeto	**object**

Algunas características de los literales más comunes

Los literales se manejan prácticamente igual que en Java. Los recordamos:

Enteros*:*

hexadecimal, se antepone el prefijo **0x**. Por ejemplo 0xFF.

binario con el prefijo **0b**, por ejemplo 0b1001 sería 9 en decimal.

Reales*:* 15.21, 3.02e10

Carácter: 'A', '4'. Secuencias de escape comunes.

Carácter	Código de escape Unicode	Código de escape especial
Comilla simple	\u0027	\'
Comilla doble	\u0022	\"
Nueva línea	\u000A	\n
Tabulación horizontal	\u0009	\t
Barra invertida	\u005C	\\

Cadena:

Uso de @ para evitar caracteres de escape.

```
Console.WriteLine("Un \ttabulador\nlínea distinta");
Console.WriteLine(@"Un \ttabulador\nlínea distinta");
```

Variables

En C# **todos los tipos de datos son objetos** (herencia común de *object*).

Las variables valor se pueden manejar de forma similar a los primitivos de java. Directamente contienen el valor, no una referencia al mismo.

Las variables referencia sí contienen una referencia al dato.

Sólo hay dos por referencia en el lenguaje base: **object y string.**

Declaración de variables:

```
byte a;
char car = 'A';
int b, num1;
long num2 = 152, num3 = 998792;
```

Tipos "nullable" : int?, float?, bool?, etc.

```
int c = null; //Error!!!
int? a = null;
float? b = null;
```

Constantes

```
const double pi=3.14159265;

const float pi=3.14F;
```

Subfijos habituales:

L long.

F float.

Operadores

Aritméticos:

```
+     suma
-     resta
*     multiplicación
 /    división
%     resto
=     asignación
```

Lógicos:

```
&&    And (Se puede usar & que evalúa ambos resultados)
||    Or (Se puede usar | que evalúa ambos resultados)
!     not
^     xor
==    "igual que"
!=    "distinto de"
<,>,<=, >=  menor, mayor, menor o igual, mayor o igual respectivamente
```

Bit a bit:

```
&     and
|     or
~     not (AltGr+4)
^     xor
<<    desplazamiento a la izquierda
>>    desplazamiento a la derecha
```

Asignación:

```
=, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>= , ++, --
```

De concatenación de cadenas y caracteres:

```
+, +=
```

Preincremento y postincremento

b=a++, primero se asigna y luego se incrementa a (postincremento).

b=++a, primero se incrementa y luego se asigna (preincremento).

Ejemplo:

```
int i = 0;
do
{
    Console.WriteLine(i++);
} while (i < 5);

Console.WriteLine();

i = 0;
do
{
    Console.WriteLine(++i);
} while (i < 5);
```

Conversión de tipos

Casting:

```
int a;
long b = 13;
a = (int)b; // Lanza excepción si no se pudiera castear


// Solo para variables referencia.
// Si no se puede castear devuelve null
MiObjeto o = otroObjeto as MiObjeto;
```

Conversión de cadenas a otros tipos:

Clase **Convert:** *Si el string es null devuelve 0.*

```
int b = Convert.ToUInt32("121");
```

Método **parse**: Si el string es null lanza **ArgumentNullException**.

```
long b = long.Parse("121");
```

Números aleatorios

```
Random generador = new Random();
double d;
int i;
//Nº real entre 0 y 1 (excluido)
d = generador.NextDouble();
//Nº entero entre 1 y 7 (excluido, es decir, de 1 a 6)
i = generador.Next(1, 7);
//Nº entero menor que 7 (excluido, es decir, de 0 a 6)
i = generador.Next(7);
```

Nunca debe meterse el **new Random() en un bucle.**

Declaración mediante var y dynamic.

```
var a = 10; // tipado implícito.
int b = 10; //tipado explícito.
```

Una vez que a una variable var se le asigna un dato el tipo queda fijado y ya no puede cambiar. Solo de be usarse si es claro el tipo que se le asigna.

```
dynamic c = "hola";
c = 44;
c++;
```

Evita el uso de dynamic, se suele reservar para tareas complejas y muy concretas.

E/S en Proyectos de Consola

Lectura de datos de usuario en consola:

Console.ReadLine()

```
string nombre = Console.ReadLine();
```

Se necesita convertir a otros tipos de datos (equivalente a nextLine() en Java)**.**

Console.ReadKey()

Lectura de tecla, puede evitarse el eco de la misma en la consola.

Escritura de datos en consola:

Console.WriteLine(dato) o **Console.Write(dato)**.

Permite cadenas con formato:

```
        int a = 10, b = 3, totalsuma, totalresta;
        string moneda;


        totalsuma = a + b;
        totalresta = a - b;
        moneda = "Euros";

         Console.WriteLine("El valor {0} sumado con el {1} resulta un total
de {2} {3}. Pero dicho valor {0} si se le resta {1} queda un total de {4}
{3}\n", a, b, totalsuma, moneda, totalresta);

       Console.WriteLine("Para mostrar {llaves}, si no hay ninguna variable
para mostrar, no hay problema.");

       Console.WriteLine("Si hay alguna, se usan dobles llaves\nLlaves:{{}}
Moneda: {0}", moneda);
```

Formato de decimales:

{modificador, nºdigitos:.000} Tantos 0 como cifras decimales

```
double pi = 3.14159265;
Console.WriteLine("{0,10:.000}", pi); // También es válido "{0,10:F3}"
```

String.Format que devuelve la cadena con formato

Formatos:

http://msdn.microsoft.com/es-es/library/txafckwd(v=vs.80).aspx

https://docs.microsoft.com/es-es/dotnet/api/system.string.format?redirectedfrom=MSDN&view=net-5.0#overloads

Control de flujo

Las sentencias selectivas **if, if-else** son idénticas a Java. También **la ternaria.**

Las estructuras repetitivas **while, do-while** y **for** también son idénticas a las equivalentes de Java.

Las clausulas **break, return** y **continue** funcionan igual que en Java.

Operadores de comprobación de null

El **operador ?** También tiene la utilidad de comprobar si un objeto es null. La sentencia:

```
miObjeto?.funcion();
```

Es equivalente a:

```
if (miObjeto != null)
{
    miObjeto.funcion()
}
```

E **operador ??** para comprobar resultados null. Por ejemplo la sentencia:

```
int b = a ?? -1;
```

Guarda en b el valor de a, salvo que a sea null, en ese caso es -1 (similar a una ternaria).

Otro uso de ?? es con asignación (**??=**):

```
variable ??= expression;
```

Es equivalente a:

```
if (variable is null)
{
    variable = expression;
}
```

Es decir, si algo no apunta a una instancia o valor se inicializa.

Más información en:

https://docs.microsoft.com/es-es/dotnet/csharp/language-reference/operators/null-coalescing-operator

https://www.csharp411.com/c-operator/

Sentencia switch

Es similar al switch de Java pero dispone de más funcionalidades:

```
        string nom;

        Console.WriteLine("Por favor introduce tu nombre");
        nom = Console.ReadLine();

        switch (nom)
        {
            case "adios":
            case "Adios":
            case "ADIOS":
                Console.WriteLine("Hasta la próxima");
                break;
            case "Curro":
                Console.WriteLine("Acceso denegado, cámbiate el nombre");
                goto case "ADIOS";
            default:
                Console.WriteLine("Hola {0}, bienvenido a este programa",
nom);
                break;
        }
```

Otras posibilidades:

```
int edad = Convert.ToInt32(Console.ReadLine());
switch (edad)
{
    case int a when (a < 0):
        Console.WriteLine("No has nacido");
        break;
    case int a when (a>= 0 && a<= 18):
        Console.WriteLine("Menor de edad");
        break;
    default:
        Console.WriteLine("Mayor de edad...supuestamente ;-)");
        break;
}
```

C#9 en adelante:

```
switch (edad)
{
    case < 0:
        Console.WriteLine("No has nacido");
        break;
    case >= 0 and < 18:
        Console.WriteLine("Menor de edad");
        break;
    default:
        Console.WriteLine("Mayor de edad...supuestamente ;-)");
        break;
}
```

Puedes ver más ejemplos en:

https://www.dotnetperls.com/switch

https://stackoverflow.com/questions/20147879/switch-case-can-i-use-a-range-instead-of-a-one-number

Métodos

Métodos con o sin parámetros y con o sin valor devuelto funciona igual que en Java.

Parámetros de salida o parámetros por referencia.

Ref:

```
static void suma (int a, int b, ref int c){
    c=a+b;
}


static void Main()
{
    int res=0;  // Obligatorio inicializar res antes de pasarlo
    suma(12, 34, ref res);
    Console.WriteLine("Resultado: {0}", res);
}
```

Out:

```
static void suma (int a, int b, out int c){
    c=a+b;
}


static void Main()
{
    int res; // out indica que res se inicia dentro del método
    suma(12, 34, out res);
    Console.WriteLine("Resultado: {0}", res);
}
```

Parámetros con nombre y opcionales

```
static void suma10(int a, int b = 10)
{
    Console.WriteLine(a + b);
}
```

Main:

```
suma10(3, 4);
suma10(b:4,a:3);
suma10(3);
```

Devolver varios valores (tuplas)

```
static (int, int, int) Operaciones (int a, int b)
{
    return (a + b, a - b, a * b);
}

static void Main(string[] args)
{
    var res = Operaciones(1, 2);
   // En lugar de var, se puede poner (int,int,int)

    Console.WriteLine("Sum:{0,4}\nRes:{1,4}\nMul:{2,4}",
        res.Item1, res.Item2, res.Item3);
}
```

Con nombres explícitos:

```
static (int suma, int resta, int multiplicacion) Operaciones (int a, int b)
{
      return (a + b, a - b, a * b);
}
```

Acceso:

```
Console.WriteLine("Sum:{0,4}\nRes:{1,4}\nMul:{2,4}", res.suma, res.resta,
      res.multiplicacion);
```

Orientación a objetos

Clase clásica :

```
using System;

class Perro
{
    public string raza;
    public string nombre;

    private int edad;

    // Nota: aunque lo veas así en este ejemplo,
    // en C# NO haremos el set y el get de esta forma.
    // La correcta se explica en un apartado posterior.
    public int getEdad()
    {
        return edad;
    }

    public void setEdad(int edad)
    {
        this.edad = edad;
    }

    public Perro()
    {
        this.setEdad(0);
        this.raza = "";
        this.nombre = "";
    }
}
```

Creación y uso del objeto :

```
class Program
{
    static void Main()
    {
        Perro objPerro=new Perro();

        objPerro.raza = "Mastín";
        objPerro.nombre = "Laika";
        objPerro.setEdad(5);
        Console.WriteLine(objPerro.getEdad());
        Console.ReadLine();
    }
}
```

Está permitido:

```
Perro objPerro2 = new Perro()
{
    nombre = "Morty",
    raza = "Pug"
};

Perro objPerro3 = new(); // Esto solo desde C# 9
```

Propiedades set/get

```
private int edad;
public int Edad
{
    set
    {
        edad = value;
    }

    get
    {
        return edad;
    }
}
```

Por supuesto las funciones setEdad y getEdad las borramos.

Versión mejorada en el set:

```
private int edad;
public int Edad
{
    set
    {
        if (value < 0)
        {
            edad = 0;
        }
        else
        {
            edad = value;
        }
    }

    get
    {
        return edad;
    }
}
```

El **programa principal** quedaría así:

```
Perro objPerro = new Perro();
objPerro.raza = "Mastín";
objPerro.nombre = "Laika";
objPerro.Edad=5;
Console.WriteLine(objPerro.Edad);
Console.ReadLine();
```

Propiedades autoimplementadas

```
public int Edad { set; get;}

public int Edad { set; get;} = 5;
```

Más sobre declaración y uso de propiedades:

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/properties

Más sobre inicialización de objetos (init, required,…):

https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/object-and-collection-initializers

Herencia, sobrecarga y sobreescritura

```
class Animal
{
    private int edad;
    public string nombreCientifico;

    // Para permitir la sobreescritura se requiere virtual.
    public virtual void MuestraEdad()
    {
        Console.WriteLine(edad.ToString());
    }
}



class Perro : Animal  // Herencia con :
{
    public string raza;
    public string nombreHumano;

    public Perro()
    {
        this.nombreCientifico = "Canis Familiaris";
    }

    public void Ladrar()
    {
        Console.WriteLine("GUAU!!!");
    }

    public void Ladrar(int n)  // Sobrecarga
    {
        for (int i = 0; i < n; i++)
            Console.WriteLine("GUAU!!!");
    }
}
```

```
class Mosca : Animal // Herencia
{
    public Mosca()
    {
        nombreCientifico = "Drosophila Melanogaster";
    }

   // Sobreescritura. Requiere override (o new, Ver apéndice II tema 2)
    public override void MuestraEdad()
    {
        Console.WriteLine("Las moscas no tienen edad, pero tenemos guardado
como dato");
        base.MuestraEdad();
    }
}
```

Sobrecarga de operadores.

Operador unario:

```
    public static Punto operator ++ (Punto p1){
        p1.x++;
        p1.y++;
        return p1;
    }
```

Operador binario:

```
    public static Punto operator +(Punto p1, Punto p2)
    {
        return new Punto(p1.x + p2.x, p1.y + p2.y);
    }
```

Algunos operadores se sobrecargan por pares (por ejemplo == y !=, o < y >).

Hay operadores que no se pueden sobrecargar como &&, || o la ternaria.

Control de acceso y otros modificadores

public: El acceso no está restringido. Igual que en Java.

protected: El acceso se limita a la clase contenedora o a los tipos derivados de ella.

internal: El acceso está limitado al ensamblado actual. El default de Java.

protected internal: El acceso está limitado al ensamblado actual o a tipos derivados de la clase contenedora. El protected de Java.

private: El acceso está limitado al tipo contenedor. Igual que en Java.

private protected: El acceso se limita a la clase que contiene o a tipos derivados de la clase que contiene dentro del ensamblado actual.

Modificadores de clase:

sealed: No se permite herencia. Equivale al final en Java.

abstract: clase abstracta.

Llamadas entre constructores: this() y base().

Clase Animal:

```
    public Animal(int edad)
    {
        this.edad = edad;
    }

    public Animal()
        : this(0)
    {
    }
```

Clase Perro:

```
    public Perro(int edad, string raza, string nombre_humano)
        : base(edad)
    {
        this.raza = raza;
        this.nombreHumano = nombre_humano;
        this.nombreCientifico = "Canis Familiaris";
    }

    public Perro()
        : this(0, "", "")
    {
    }
```

Polimorfismo

Detectar que un objeto es de **determinado tipo o heredado** con el comando **is** (similar al *instanceof* de Java).

```
static void Main() {
    Perro objPerro = new Perro(2, "Mastin", "Neo");
    Animal objAnimal;

    objAnimal = objPerro;
    Console.WriteLine(objAnimal.nombreCientifico);
    if (objAnimal is Perro)
    {
       Console.WriteLine(((Perro)objAnimal).nombreHumano);
    }

    Console.ReadKey();
}
```

Para comprobar el **tipo exacto** se debe usar el método **GetType()** definido en la clase **object** y el comando **typeof()** de la siguiente forma (equivalente el uso de getClass() de Java):

```
if (objAnimal.GetType()==typeof(Perro))
{
    Console.WriteLine(((Perro)objAnimal).nombreHumano);
}
```

Uso de **switch para resolver el polimorfismo** y hacer el casting al mismo tiempo:

```
switch (objAnimal)
{
    case Perro p:  // Si p no es necesario llega case Perro:
        Console.WriteLine(p.nombreHumano);
        break;
    case Mosca m:
        Console.WriteLine("Soy la Mosca!");
        m.MuestraEdad();
        break;
}
```

Variables de sólo lectura

```
public readonly static string definicion = "ser vivo eucariota
pluricelular";
```

Interfaces

Definición:

```
interface ICorredor
{
    void correr();
}
```

Implementación:

```
class Perro : Animal, ICorredor {...

class Coche : ICorredor {...
```

Arrays

Unidimensionales

```
int[] v = new int[10];
Random g = new Random();
Console.WriteLine("El tamaño del vector es {0}", v.Length);

for (int i = 0; i < v.Length; i++)
{
    v[i] = g.Next(-5,5);
}

for (int i = 0; i < v.Length; i++)
{
    Console.WriteLine("Posición: {0,2}\tValor: {1,3}", i, v[i]);
}
```

Multidimensionales

```
int i, j;
int[,] bi = new int[3, 5]; // OJO, cambia respecto a Java.
Random g = new Random();

Console.WriteLine("Array bidimensional");
for (i = 0; i < bi.GetLength(0); i++)       // Cantidad de filas
{
  for (j = 0; j < bi.GetLength(1); j++)     // Cantidad de columnas
  {
          bi[i, j] = g.Next(1,21);
  }
}

for (i = 0; i <= bi.GetUpperBound(0); i++)    // Limite superior de filas
{
   for (j = 0; j <= bi.GetUpperBound(1); j++) //Limite superior de columnas
   {
          Console.Write("{0,3}", bi[i, j]);
   }
   Console.WriteLine();
}
Console.WriteLine("El nº de dimensiones del array es {0}", bi.Rank);
```

Inicialización a valores predeterminados

```
int[] vector = { 5, 1, 4, 0 };

int[,] tabla = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 } };
```

Tablas dentadas

Equivalentes a arrays bidimensionales en Java, se usan sólo para filas de distinto tamaño:

```
Random g = new Random();
int[][] tabla = new int[3][];
tabla[0] = new int[4];
tabla[1] = new int[2];
tabla[2] = new int[5];

for (int i = 0; i < tabla.Length; i++)
{
    for (int j = 0; j < tabla[i].Length; j++)
    {
        tabla[i][j] = g.Next(10, 40);
    }

}
```

Bucle foreach

```
foreach(int dato in vector)
{
    Console.Write("{0,4}", dato);
}
Console.WriteLine();
```

¡En bidimensional lo recorre entero!

Cadenas (string). Inmutables.

```
string nombre;
Console.Write("Introduce tu nombre: ");
nombre = Console.ReadLine();
if (nombre == "Curro")  // OJO, se compara con ==
{
    Console.WriteLine("No quiero hablar contigo");
}
else
{
    Console.WriteLine("¡¡¡Bienvenido!!! :-)");
}
```

Sin tener en cuenta mayúsculas o minúsculas puedo usar **ToUpper, ToLower o sobrecarga de Equals.**

```
if (nombre.ToUpper()=="CURRO")

if (nombre.Equals("Curro",StringComparison.CurrentCultureIgnoreCase))
```

Indexación con []:

```
Console.WriteLine(nombre[0]);
```

Si se requiere **mutabilidad**, usa la clase **StringBuilder.**

Algunas propiedades y métodos de la clase String (y Char).

Remove(int pos, int n) *:* elimina subcadenas de *n* caracteres a partir de pos. Devuelve la cadena modificada.

Insert(int pos, string cad): Inserta *cad* a partir de *pos.* Devuelve la cadena modificada.

Replace(Cadena a Sustituir, sustituta): Devuelve la cadena resultante de sustituir en la cadena sobre la que se aplica toda aparición de la cadena a sustituir indicada por la cadena sustituta especificada como segundo parámetro.

Lenght: Devuelve la longitud de la cadena.

EndsWith(cadena): Función booleana. Comprueba si la cadena acaba por el parámetro

StartsWith(cadena): Función booleana. Comprueba si la cadena empieza por el parámetro

Substring(posición inicial, longitud): Función que devuelve una subcadena.

Split(array de char): devuelve un array de cadenas tras "romper" la original por los caracteres indicados en el array.

ToUpper, ToLower: Funciones que devuelven el string pasado a mayúsculas y minúsculas respectivamente. (Tiene en cuenta configuración regional).

Trim(): Función que quita espacios al principio y final de la cadena y la devuelve. Existe *TrimEnd* y *TrimStart* para quitar los espacios solo del final o del principio respectivamente. Devuelve la cadena modificada.

Funciones estáticas:

string.Format(cadena): da formateo igual que se vio para WriteLine pero devolviendo la cadena formateada.

string.IsNullOrEmpty(cadena): Comprubea si el string es null o es una cadena vacía "" .

Char.IsDigit(char c): Comprueba si c es un dígito numérico

Char.IsLetter(char c): Comprueba si c es una letra.

Char.IsPunctuation(char c): Comprueba si c es un signo de puntuación.

Cadenas interpoladas

Se antepone $ y se usan las variables directamente entre {}.

```
int[] v  = { 5, 1, 4, 0 };
DateTime date = DateTime.Now;

//Uso var para evitar el using y una declaración excesivamente larga
var culture = new System.Globalization.CultureInfo("es-ES");
String day = culture.DateTimeFormat.GetDayName(date.DayOfWeek);

Console.WriteLine($"¡Hola! Hoy es {day}, y son las {date:HH:mm}.");
Console.WriteLine("Veamos el contenido de un vector");
for (int i = 0; i < v.Length; i++)
{
    Console.WriteLine($"Posición: {i,3}\tValor: {v[i],3}");
}
```

Algunas sobrecargas de ToString.

```
int num = 60;

// 3 decimales
Console.WriteLine((num/7.0).ToString("F3"));

// Hexadecimal con 4 dígitos
Console.WriteLine(num.ToString("X4"));

// Para otras bases se usa Convert.ToString
Console.WriteLine(Convert.ToString(num, 2)); //Binario
Console.WriteLine(Convert.ToString(num, 8)); //Octal

// Formatos según configuración regional
Console.WriteLine(DateTime.Now.ToString(
    CultureInfo.CurrentCulture.DateTimeFormat.LongDatePattern));

Console.WriteLine(DateTime.Now.ToString(
    CultureInfo.CurrentCulture.DateTimeFormat.ShortDatePattern));
```

Más en:
https://learn.microsoft.com/es-es/dotnet/standard/base-types/standard-numeric-format-strings

Colecciones: List<T>

A continuación indicamos algunos métodos y propiedades de estas colecciones:

Count: Cantidad de elementos.

Add(elemento): Añadir un elemento (al final de la colección).

RemoveAt(posición entre 0 y Count-1): Eliminar un elemento en la posición indicada

Remove(objeto): Para eliminar cierto objeto.

Insert(nº de posición, elemento): Insertar un elemento en la posición indicada

Clear(): Borra todo el contenido de una colección

Contains(elemento): Devuelve true si el elemento se encuentra en la colección

IndexOf(elemento): Devuelve el índice del elemento (la primera vez que aparece) o -1 si no existe.

Sort: Ordena la colección. Ojo, da error si la colección contiene objetos de distintos tipos.

Ejemplo:

```
List<int> colEnteros = new List<int>();
Random g = new Random();

for (int i = 0; i < 5; i++)
{
    colEnteros.Add(g.Next(0, 10));
}

colEnteros.Sort();
colObjetos.RemoveAt(0);
```

```
foreach (int num in co colEnteros)
{
    Console.Write($"{num, 5}");
}
Console.WriteLine();
```

Colección Arraylist

Existe pero es genérica de elementos tipo object.

```
ArrayList colObjetos = new ArrayList();
colObjetos.Add("Isaac Asimov");
colObjetos.Add(231);
colObjetos.Add(Math.PI);
colObjetos.Add("Robert A. Heinlein");
foreach (object s in colObjetos)
{
      if (!(s is string))
      {
          Console.WriteLine($"{s,6:F2}");
      }
}
```

Tablas Hash (diccionario)

```
Dictionary<String, int> edades = new Dictionary<string, int>();

      // Introducir elementos (calve, valor)
      edades.Add("Ana", 20);
      edades.Add("Juan", 31);
      edades.Add("Pablo", 2);
      edades.Add("Maria", 12);

      // Mostrar un elemento mediante hashing
      Console.WriteLine("Ana tiene {0} años", edades["Ana"]);

      // Recorrer la colección pares clave/valor
      foreach (KeyValuePair<String,int> de in edades)
      {
          Console.WriteLine("{0} tiene {1} años", de.Key, de.Value);
      }

      // Recorrer la colección solo de valores
      foreach (int de in edades.Values)
      {
          Console.WriteLine(de);
      }

      // Recorrer la colección solo de claves
      foreach (string de in edades.Keys)
      {
          Console.WriteLine(de);
      }
```

Con Hashtable:

```
        Hashtable edades = new Hashtable();
```

y el primer bucle sería:

```
        // Recorrer la colección pares clave/valor
        foreach (DictionaryEntry de in edades)
        {
            Console.WriteLine("{0} tiene {1} años", de.Key, de.Value);
        }
```

Excepciones

```
do
{
    try
    {
        error = false;
        Console.WriteLine("Por favor, introduce un número");
        num = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine($"Has introducido el nº {num}");
    }
      // Si no se usa el objeto excepción no es necesario definirlo
    catch (FormatException)
    {
        Console.WriteLine("Ha introducido un valor no entero");
        error = true;
    }
    catch (OverflowException e)
    {
        Console.WriteLine($"Número inválido: {e.Message}");
        error = true;
    }
    catch // Si no se pone tipo de excepción es Exception
    {
        Console.WriteLine("Error no contemplado, el programa finalizará");
        Environment.Exit(-1);
    }
} while (error);
```

Lanzamiento de excepciones con throw:

```
      if (num < 0)
      {
          throw new System.ArgumentOutOfRangeException();
      }
```

Se puede relanzar la misma excepción en un catch:

```
      catch (FormatException)
      {
         Console.WriteLine("dato invalido”);
         throw;
      }
```

Excepción con catch condicional:

Se captura si además de saltar dicha excepción cumple la condición.

```
catch (ArgumentOutOfRangeException) when (num < 10000)
{
    Console.WriteLine("No es un número valido");
    error = true;
}
```

Capturar varias excepciones que hacen lo mismo:

```
catch (Exception ex) when ( ex is FormatException || ex is … || ex is …)
```

TryParse

```
bool tipo.TryParse(string s, out tipo numero);
```

Ejemplo:

```
int n;
bool bandera;
do
{
    Console.WriteLine("Dime un nº");
    bandera = int.TryParse(Console.ReadLine(), out n);
    if (!bandera)
    {
       Console.WriteLine("No es un entero, vuelve a intentarlo");
    }
} while (!bandera);

Console.WriteLine($"¡Bien! tu número introducido es:{n,2}");
```

Solo comprobación de dato válido:

```
if (int.TryParse(Console.ReadLine(), out _))
```

Enumeraciones

```
enum Dias
{
    Lunes = 1,
    Martes,
    Miercoles,
    Jueves,
    Viernes,
    Sabado,
    Domingo
}

enum Bancos
{
      Abanca = 2080,
      Bankia = 3922,
      BBVA = 1020,
      Bankinter = 3021,
      BSCH = 4231,
      Banesto = 3489,
      Caixavigo = Abanca
}
```

En el main:

```
Dias dia;
dia = Dias.Miercoles;
dia++;
Console.WriteLine(dia); // Verlo como enumerado

// Verlo como número
Console.WriteLine((int)dia);
Console.WriteLine(dia.ToString("D")); // decimal

dia = (Dias)3;
Console.WriteLine(dia);
```

Algunos métodos estáticos de la clase Enum:

- **Enum.GetNames():**

```
string[] dias = Enum.GetNames(typeof(Dias));
int i = 0;
foreach(string d in dias)
{
      Console.WriteLine($"{++i} .- {d}") ;
}
```

- **static bool isDefined (Type enum, object valor)**: Devuelve un booleano que indica si algún literal de la enumeración indicada tiene el valor indicado.

- **static object Parse(Type enum, string nome, [bool mayusculas])**: Para **convertir una cadena a enumerado** si hay correspondencia. Muy práctica para guardar enumerados como cadenas en archivos, bases de datos o en componentes como ComboBoxes.

  En caso de que no se pueda convertir salta excepción.

```
dia = (Dias)Enum.Parse(typeof(Dias), "Viernes", true);

Console.WriteLine(dia);
```

  OJO: No es recomendable usarlo para pedir enumerados al usuario, pues debería escribirlo completo, al usuario siempre un menú o combobox en modo gráfico.

Clases indexadas

```
class Prueba
 {
     private int[] v = new int[100];

     public int this[int indice]
     {
         set
         {
             v[indice] = value;
         }
         get
         {
             return v[indice];
         }
     }

     // Otros métodos y propiedades de la clase Prueba

 }
 class Program
 {
     static void Main()
     {
         Prueba p = new Prueba();
         p[3] = 12;
         Console.WriteLine(p[3]);
         Console.ReadKey();
     }
 }
```

Tipos genéricos

Método genérico:

```
public static void MostrarElemento<T>(T elemento)
{
    Console.WriteLine($"El elemento es: {elemento}");
}

static void Main(string[] args)
{
    MostrarElemento<int>(42);           // Con entero
    MostrarElemento<string>("Hola");    // Con string
    MostrarElemento<double>(3.14);      // Con double
}
```

Clase genérica:

```
class CajaDoble<T1, T2>
{
    public T1 Item1 { set; get; }
    public T2 Item2 { set; get; }
    public CajaDoble(T1 item1, T2 item2)
    {
        this.Item1 = item1;
        this.Item2 = item2;
    }
    public void MostrarContenido()
    {
      Console.WriteLine($"Esta caja contiene un {Item1} y un {Item2}");
    }

    public T1[] ToArray()
    {
        if (Item2 is T1 aux )
        {
            return new T1[] { Item1, aux };
        }
    throw new InvalidCastException(
       "No se puede convertir Item2 a  T1");
    }
}
```

```
internal class Program
{
    static void Main(string[] args)
    {
      var caja1 = new CajaDoble<string, string>("Manzana", "Naranja");
      caja1.MostrarContenido();
      Console.WriteLine(
       $"Array: [{caja1.ToArray()[0]}, {caja1.ToArray()[1]}]");
    }
}
```

Ejemplo real: la clase List<T>.

https://github.com/microsoft/referencesource/blob/main/mscorlib/system/collections/generic/list.cs